Formal Verification of the Custodian Daemon by OG Technologies EU

Stellar Go CLI Security Assessment by Runtime Verification / Specula Lite / TLA+
Analysis of `internal/custodian`

stellar-go-cli

Specula Lite workflow:
automated model synthesis, TLC model checking,
counterexample-to-code mapping, remediation, re-verification

November 2026

Abstract

We applied the Specula Lite methodology — a lightweight, single-agent variant of the Specula formal-verification pipeline — to the custodian signing daemon of stellar-go-cli (`internal/custodian/`), the component that owns key custody, enforces the operator policy (E1–E5), and brokers all passkey-protected and mandate-delegated signatures. A hand-audited TLA+ model of the enforcement state machine (660 lines, file:line-annotated to the Go source) was exhaustively model checked with TLC. All seven targeted hunt invariants produced counterexamples, every one of which mapped to a concrete implementation defect — eleven confirmed findings (F1–F11) including two rated **high** severity: an unguarded agent reactivation path and unsynchronized maps that produce an unrecoverable `fatal error: concurrent map writes` remote crash. Each finding was reproduced by an executable Go test. All findings were subsequently remediated; the model was updated to match the fixed implementation and the hunts re-run: four invariants now pass exhaustively at their bounds, one remains violated by design (a documented capacity bound), and two show only a microsecond-scale residual revocation window. The full test suite, race detector, `go vet`, and `golangci-lint` pass on the remediated tree.

Contents

1 Introduction **3**

2 Scope and Methodology **3**
2.1 Toolchain 3
2.2 System category and abstractions 4
2.3 Limits of this analysis 4

3 Model Overview **4**

4 Findings **5**
4.1 Summary 5
4.2 Commit boundary (F1–F3) 5
4.3 Concurrency (F7, F8) 6

4.4 Escalation limiter (F4) . . . 6
4.5 Authorization (F5, F9, F10) . . . 6
4.6 Persistence (F6, F11) . . . 6
4.7 Hygiene . . . 7

5 Verification Results **7**
5.1 Model-checking runs . . . 7
5.2 Residual bounds . . . 7
5.3 Go-level verification . . . 7

6 Model Artifacts Kept Honest **8**

7 Recommendations **8**

A Reproduction Commands **8**
A.1 File inventory of the remediation . . . 8

1 Introduction

The custodian daemon (`internal/custodian/`, ≈5,000 LOC) is the security-critical core of the agent-facing signing architecture: it holds passkey unlock grants in memory, enforces the operator policy (`state/policy.json`) through the E1–E5 pipeline, brokers WebAuthn ceremonies, manages mandate escrow for delegated agents, and appends every decision to a hash-chained enforcement log. Because requests arrive over a Unix socket on per-connection goroutines, correctness depends on subtle ordering and synchronization invariants that unit tests struggle to cover — precisely the class of defects model checking is designed to find.

This report documents a complete verification cycle:

1. **Model synthesis** — a TLA+ specification of the enforcement state machine, faithful to real lock boundaries.
2. **Targeted hunts** — seven invariants checked under counter-bounded environment actions; all seven produced counterexamples.
3. **Code mapping & reproduction** — each counterexample mapped to concrete source lines and reproduced by a deterministic Go test.
4. **Remediation** — all findings fixed in the implementation.
5. **Re-verification** — model updated to match the fixes; hunts re-run; regression suite asserts the new behavior.

Headline results

- 7/7 hunt invariants violated pre-fix ⇒ 11 confirmed findings (F1–F11): 2 **high**, 4 medium, 5 low severity.
- Post-fix re-verification: 4 invariants **PASS** exhaustively; 1 **expected violation** (accepted capacity bound, F11); 2 **residual violations** confined to a microsecond commit-tail window (F9 remainder).
- Convergence run on the updated model: **20,032,814** states generated, **6,005,402** distinct, depth 22, full queue exhausted, no invariant violations.
- 15 `TestRepro_*` regression tests assert the fixed behavior and go red if a defect returns.

2 Scope and Methodology

2.1 Toolchain

Specula Lite is the officially supported lightweight path of the Specula project: it preserves the agentic spec-synthesis → model-checking → code-level reproduction loop while requiring no Specula installation, MCP server, Maven, or agent adapter — only a JRE and `tla2tools.jar`, bootstrapped into the per-user cache (`.cache/specula-lite/`). Checking was performed with TLC (TlaTools 1.8.0) under a bundled JRE 21:

```
java -Xmx3g -XX:+UseParallelGC -cp tla2tools-1.8.0.jar \
  tlc2.TLC -deadlock -workers 4 -cleanup MC
```

All artifacts live in the (gitignored) `.specula-lite/` directory:

- `spec/base.tla` — the model; `spec/MC.tla` — checking wrapper
- `spec/MC.cfg` + seven `spec/MC_hunt_*.cfg` — run configs
- `modeling-brief.md`, `analysis-report.md`, `report.md` — methodology, audit trail, results

The durable review summary is `docs/audit/specula-lite-custodian-findings.md`.

2.2 System category and abstractions

The daemon is a *Category A* (message-passing) system: client ↔ Unix socket ↔ per-connection handler goroutines over in-memory stores, TTL-limited authority, and crash/restart with log-based rebuild. The specification splits the E1–E5 pipeline at every real mutex/store boundary — the same points where Go interleavings occur — rather than collapsing the pipeline into one step. Key abstractions:

- WebAuthn ceremonies = atomic verify-or-fail oracles (the passkey itself is trusted hardware, not the system under analysis).
- XDR decode, Horizon I/O, socket-token check = environment.
- Request payloads are well-formed; refusals of interest are authority/escalation/replay/cap refusals, not parse errors.
- The >10k-entry seen-set aging gate is modeled as pure time-based eviction (sound: the trace is valid under the entry-pressure it documents).

2.3 Limits of this analysis

- *Lite mode* omits Specula's trace-validation harness phase; spec↔code correspondence rests on the file:line-annotated model plus the executable reproduction tests.
- State space is bounded (2 in-flight requests, 1–2 hashes/refs, MaxT 2–5, small caps) — finite by construction and fully exhausted in every reported run. Violations at these bounds are real; absence of violations is proven within the bounds, not beyond.
- `sign_credential` uses a separate lighter pipeline and is out of scope.

3 Model Overview

Component	Modeled semantics
Grant store	wallet unlock grants + mandate escrow, TTL expiry, lock/drop
Approval store	single-use bound approvals keyed by request hash, TTL
Seen set (`decisions`)	single-use decision refs, claim/release, time aging
Pending queue (`pendingReq`)	bounded E3-escalation tracking, lazy expiry
Escalation limiter	sliding-window hit budget per account
Spend counters	global daily caps and mandate cumulative caps, persisted
Mandate registry	VC lifecycle: none/active/suspended/revoked
Agent registry	enrolled→active→suspended→retired lifecycle
`issued` set	payload hashes the daemon signed, eligible for submit
`submit_tx`	atomic claim of an issued hash, Horizon result
Restart	memory wipe; decisions/issued rebuilt from the log

Environment actions are counter-bounded in `MC.tla` (unlock, lock, disable, confirm, agent and mandate status, issue, restart, evict); pipeline stages and submit steps are unbounded so the real interleavings remain reachable.

4 Findings

4.1 Summary

ID	Severity	Defect	Fix
F1	low	`issued` marked before the decision-ref claim; refused signs left a submittable blob	pure sign stage; mark moved inside the commit boundary
F2	med	charge-step refusal burned the single-use decision ref	`decisions.release` on charge failure
F3	med	multi-op partial charge committed earlier ops on refusal	atomic `chargeAll` on both counters + refund
F4	med	configured escalation limiter rebuilt per call — never engaged	persistent `configuredLimiter` under `limiterMu`
F5	**high**	`agent_set_status` reactivated suspended→active with only the socket token	bound `agent_activate` ceremony required
F6	low-med	restart resurrected `issued` for already-submitted payloads	`payload_hash` on submit records; sequential tombstone replay
F7	med	`submit_tx` check-then-delete race → double submit	atomic `issuedTake` under `issuedMu`
F8	**high**	`issued`/`pendingReq` unsynchronized maps → runtime-fatal concurrent-map crash	`issuedMu` + `pendingMu` guard all access
F9	low	authority TOCTOU: revocation between E2 sampling and delivery	`authorityRevoked` re-check immediately before `sign`
F10	low	`confirm` minted approvals for any hash — escalation funnel advisory	confirm requires a tracked `pendingReq` entry
F11	low	seen-set aging (>10k/24h) allows decision-ref replay	**accepted bound** — documented; mitigated by F2 fix

4.2 Commit boundary (F1–F3)

Defect. `enforce.go` produced the signature and marked `issued[sigHash] = true` *before* the single-use `decisions.claim` and the per-op charge loop. The TLC s1 trace shows a request refused at claim retaining `rMut = {issued}`; the charge variant retains `{issued, claimed, charged}` with counters incremented.

Fix. Commit boundary reordered to: authority re-check → `d.sign` (pure output) → `claim` → `chargeSpend` (atomic `chargeAll` on mandate and global counters; global failure refunds the mandate charge) → `issuedMark`. A charge refusal calls `decisions.release`. Refused requests now commit nothing — verified by `TestRepro_RefusedClaimLeavesNoResidue`, `TestRepro_RefSurvivesChargeRefusal`, and `TestRepro_ChargeIsAtomic`, plus unit tests of `chargeAll` and `refund`. TLC: **RefusePure passes** (312,666 states, exhaustive).

4.3 Concurrency (F7, F8)

Defect. `d.issued` and `d.pendingReq` were plain maps accessed by per-connection goroutines while every neighboring store was mutex-guarded. A race-hammer test produced `fatal error: concurrent map writes` — an unrecoverable runtime fatal, i.e., any client able to open two connections could crash the custodian and drop every live grant (fail-closed availability violation). Separately, `submitTx`'s check-then-delete let two submitters both pass the issued check.

Fix. `issuedMu` and `pendingMu` guard every access; `issuedTake` performs check+delete inside one critical section. Verified by `TestRepro_IssuedTakeIsAtomic` (sequential + 8-goroutine single-winner hammer) and `TestRepro_RaceHammer` under `-race`. TLC: **SubmitOnce passes**.

4.4 Escalation limiter (F4)

Defect. `limiterFor()` constructed a *new* `escalationLimiter` on every call whenever `policy.Escalation.Rate` was configured, so `allow()` always saw an empty hit window and the configured rate never engaged.

Fix. The configured limiter is built once under `limiterMu` and reused. Verified by `TestRepro_ConfiguredLimiterEngages` (count=1 refuses the second escalation; identical instance returned). TLC: **PendingBound passes** (812,713 states).

4.5 Authorization (F5, F9, F10)

F5 — unguarded agent reactivation (high). `agent_set_status` accepted `suspended→active` (and `enrolled→active`) with only the socket token — asymmetric with mandate reactivation, which requires a relayed principal ceremony. A token-holding agent could un-suspend itself. *Fix:* every transition to `active` routes through a bound `agent_activate` WebAuthn ceremony; `AgentSetStatus/signer agent reactivate` gained an `--account` flag selecting the operator wallet whose passkey authorizes it. Privilege reductions remain ceremony-free.

F9 — authority TOCTOU. The pipeline sampled grant/escrow/mandate/agent authority once at E2; revocation mid-request did not stop delivery of the post-revocation signature. *Fix:* `authorityRevoked` re-validates all mutable inputs immediately before `d.sign`. See §5.2 for the remaining commit-tail window.

F10 — blind confirms. `completeApproval` minted a bound approval for whatever `requestHash` the client supplied — the escalation funnel (limiter + pending queue + operator triage) was advisory. *Fix:* `MethodConfirm` refuses hashes absent from `pendingReq`; the operator can only approve requests the daemon actually escalated. TLC: **ApprovalBound passes** (754,688 states).

4.6 Persistence (F6, F11)

F6 — restart resurrection. `rebuildIssued` restored `issued` for every `signed:` log record; `submitted:` records (which carried the Horizon tx hash, not the payload hash) did not tombstone it, so a payload submitted pre-restart passed the E5 check again post-restart. *Fix:* submit records now carry `payload_hash`; rebuild replays the log sequentially — `signed:` adds, `submitted:` tombstones, and a legitimate re-sign re-adds. Verified by `TestRepro_SubmittedTombstonedOnRestart`.

F11 — seen-set aging (accepted bound). Refs older than 24h are evictable once the set exceeds 10,000 entries; an evicted ref can be re-claimed, authorizing a second signature for the same decision. *Disposition:* documented bound — practical impact is bounded and shrinks further

now that refused requests release their refs (F2). `TestRepro_SeenSetAgingBound` documents the semantics; the s3 hunt remains violated *by design* under the time-based-eviction abstraction.

4.7 Hygiene

`MethodDisable` dropped the wallet grant but left `mandate:` escrow entries (keyed by mandate ID) holding seed material in memory. *Fix:* `dropWalletGrants` sweeps every mandate escrow on `passkey_disable` and `remove_wallet` (`TestRepro_DropWalletGrantsSweepsEscrows`).

5 Verification Results

5.1 Model-checking runs

Run	Outcome	Detail
`MC.cfg` convergence	**PASS**	20,032,814 generated / 6,005,402 distinct, depth 22, queue exhausted; TypeOK + DeliveredLogged hold
s1_commit (RefusePure)	**PASS**	312,666 / 142,315, depth 18 — refused requests commit nothing
s2_submit (SubmitOnce)	**PASS**	4,514 / 2,444, depth 14 — atomic take; restart tombstones
s4_approval (ApprovalBound)	**PASS**	754,688 / 363,348, depth 20 — no blind approvals
s4_limiter (PendingBound)	**PASS**	812,713 / 351,769, depth 19 — configured limiter engages
s3_lifecycle (DecisionOnce)	**expected viol.**	F11 accepted bound: aged ref re-claimed → second sign
s5_agents (NoDirtyAgentSign)	**residual**	violation only inside the commit tail (suspend post-recheck, pre-deliver)
s6_toctou (NoStale/ NoRevokedAuthority)	**residual**	violation only inside the commit tail (lock/disable post-recheck, pre-deliver)

5.2 Residual bounds

F9 commit-tail window. The s5/s6 traces now require revocation to land *after* the pre-sign `authorityRevoked` re-check and *before* response delivery — the claim→charge→mark→return tail. In the real implementation that window shrank from the entire E2→deliver pipeline (including escalation waits, milliseconds) to microseconds of commit code. Eliminating it requires revocation to be atomic with delivery — a daemon-level read lock spanning validation→return on `grantStore/agentRegistry` (the deferred `sync.RWMutex` design). Severity: low; documented.

F11 and lazy `pendingReq` expiry are documented above.

5.3 Go-level verification

- `go test ./internal/custodian/` — **pass** (full suite).
- `go test -race ./internal/custodian/` — **pass**; the race hammer is now a permanent detector test.

- `go vet ./...`, `gofmt -l`, `golangci-lint run` — **clean**, both modules.
- 15 `TestRepro_*` regression tests assert the fixed behavior; each names its originating counterexample in comments.

6 Model Artifacts Kept Honest

- `EvictDecision` abstracts the >10k-entry aging gate as pure time eviction — s3 traces are valid under that pressure.
- `delivered`/`submitted`/`log` are observability vars; hunt cfgs hide them via `VIEW` only where neither actions nor the checked invariant read them (documented per cfg).
- Restart kills in-flight requests to `refused` (connection death); a mutated-but-refused in-flight request at restart is a real edge the model keeps visible via `rMut`.
- The ceremony path clears `agentDirty` — an operator-blessed reactivation legitimately restores trust, so post-ceremony signatures are not violations.

7 Recommendations

1. Add a `go test -race` job for `internal/custodian` to CI — the F8 class is exactly what it catches.
2. Add conformance vectors C15 ("refusals are side-effect-free") and C16 ("escalation requires tracked pending") to the C3–C14 suite.
3. If revocation promptness ever becomes a hard requirement, implement the deferred `sync.RWMutex` design spanning E2→delivery.
4. Re-run the hunts on any change to `enforce.go`, `grants.go`, `pending.go`, `submit.go`, or the agent/mandate lifecycle — the cfgs are cheap to replay (§A).

A Reproduction Commands

```
# regression suite (fixed behavior)
go test ./internal/custodian/ -run TestRepro_ -v
go test -race ./internal/custodian/
go vet ./... && golangci-lint run ./internal/custodian/...

# TLC hunts: swap each .specula-lite/spec/MC_hunt_*.cfg into MC.cfg, then
cd .specula-lite/spec
java -Xmx3g -XX:+UseParallelGC -cp tla2tools-1.8.0.jar \
  tlc2.TLC -deadlock -workers 4 -cleanup MC
```

A.1 File inventory of the remediation

- `internal/custodian/daemon.go` — `issuedMu`, `pendingMu`, `issuedMark`, `issuedTake`, `dropWalletGrants`, rebuild tombstoning, F10 confirm gate, F5 ceremony dispatch, disable sweep.
- `internal/custodian/enforce.go` — commit-boundary reorder, `authorityRevoked`, `chargeSpend`, payload-hash records.
- `internal/custodian/grants.go` — `seenSet.release`.
- `internal/custodian/pending.go` — `pendingMu` guards, persistent `configuredLimiter`.

- `internal/custodian/policy.go` — `spendCounters.chargeAll`, `subAmount`.
- `internal/custodian/mandate.go` — `mandateCounters.chargeAll`, `refund`.
- `internal/custodian/submit.go` — atomic `issuedTake`, `payload_hash` on records.
- `internal/custodian/ceremony.go` — `agent_activate` action.
- `internal/custodian/agents.go` — unchanged (transition table was already correct).
- `internal/signsock/` — `protocol.go`, `client.go`: ceremony fields, relayed `AgentSetStatus`.
- `cmd/stellar-go-cli/commands/ signer.go` — `--account` flag for reactivation.
- `internal/custodian/specula_lite_repro_test.go` — 15 regression tests.
- `.specula-lite/spec/ base.tla`, `MC.tla`, `MC_hunt_*.cfg` — updated model + VIEW projections.